Université Privée de Fès

Filière Génie Informatique – 3ème Année

Technologie Web 2

Accès aux Bases de Données avec PHP et PDO

Année Universitaire 2025–2026

Prof. Marwane

Table des matières

1 Introduction aux Bases de Données en PHP **2**
1.1 Pourquoi une base de données ? 2
1.2 Qu'est-ce que PDO ? 2

2 Création de la Base de Données MySQL **3**

3 Connexion à MySQL avec PDO **5**

4 Les Opérations CRUD avec PDO **7**
4.1 CREATE – Insérer des données 7
4.2 READ – Lire des données 8
4.2.1 Les modes de récupération (Fetch Modes) 9
4.3 UPDATE – Modifier des données 9
4.4 DELETE – Supprimer des données 10

5 Requêtes Préparées : bindValue() vs bindParam() **11**

6 Les Transactions **12**

7 Pagination des Résultats **14**

8 Système d'Authentification avec PDO **16**
8.1 Inscription sécurisée 16
8.2 Connexion sécurisée 17

9 Récapitulatif des Méthodes PDO **19**
9.1 Méthodes de l'objet PDO 19
9.2 Méthodes de PDOStatement 19
9.3 Schéma décisionnel 19

10 Bonnes Pratiques et Erreurs Courantes **20**
10.1 Erreurs courantes 20
10.2 Structure recommandée du projet 20

1 Introduction aux Bases de Données en PHP

1.1 Pourquoi une base de données ?

Jusqu'à présent, nous avons travaillé avec des données en mémoire (tableaux, variables) ou dans des fichiers texte. Ces approches ont des **limitations** : données perdues à la fin du script, pas de recherche efficace, pas d'accès concurrents, pas de relations entre les données.

Pourquoi ? – Pourquoi MySQL avec PHP ?

MySQL est le SGBDR le plus utilisé avec PHP. Il est gratuit, performant, et intégré dans XAMPP/WAMP. PHP collecte les données via les formulaires, MySQL les stocke, et PHP les récupère pour les afficher.

1.2 Qu'est-ce que PDO ?

Définition

PDO (*PHP Data Objects*) est une interface d'accès aux bases de données en PHP. C'est une couche d'abstraction fournissant une API uniforme pour MySQL, PostgreSQL, SQLite, Oracle, etc.

PHP a connu trois générations d'accès aux bases de données :

Extension	Description	Statut
`mysql_*`	Fonctions originales (PHP 2.0)	**Supprimée**
`mysqli_*`	MySQL Improved (PHP 5.0)	Active mais limitée
`PDO`	PHP Data Objects (PHP 5.1+)	**Recommandée**

Astuce – Avantages de PDO

- **Portabilité** : même code pour MySQL, PostgreSQL, SQLite, etc.
- **Requêtes préparées** : protection native contre les injections SQL.
- **Orienté objet** : code plus propre et maintenable.
- **Gestion d'erreurs** : exceptions pour un meilleur débogage.
- **Standard industriel** : utilisé par tous les frameworks (Laravel, Symfony...).

2 Création de la Base de Données MySQL

Exemple – Script SQL complet

```sql
CREATE DATABASE IF NOT EXISTS upf_etudiants
    DEFAULT CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE upf_etudiants;

CREATE TABLE IF NOT EXISTS filieres (
    id           INT AUTO_INCREMENT PRIMARY KEY,
    code         VARCHAR(10) NOT NULL UNIQUE,
    nom          VARCHAR(100) NOT NULL,
    description TEXT,
    date_creation DATE DEFAULT (CURRENT_DATE)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS etudiants (
    id           INT AUTO_INCREMENT PRIMARY KEY,
    cne          VARCHAR(20) NOT NULL UNIQUE,
    nom          VARCHAR(50) NOT NULL,
    prenom       VARCHAR(50) NOT NULL,
    email        VARCHAR(100) UNIQUE,
    date_naissance DATE,
    filiere_id   INT,
    moyenne      DECIMAL(4,2) DEFAULT 0.00,
    photo        VARCHAR(255) DEFAULT NULL,
    date_inscription DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (filiere_id) REFERENCES filieres(id)
        ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS utilisateurs (
    id           INT AUTO_INCREMENT PRIMARY KEY,
    nom          VARCHAR(50) NOT NULL,
    email        VARCHAR(100) NOT NULL UNIQUE,
    mot_de_passe VARCHAR(255) NOT NULL,
    role         ENUM('admin', 'professeur', 'etudiant')
                 DEFAULT 'etudiant',
    date_creation DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Exemple – Données de test

```sql
INSERT INTO filieres (code, nom, description) VALUES
('GI', 'Génie Informatique', 'Développement logiciel'),
('GC', 'Génie Civil', 'Construction et BTP'),
```

```sql
('GF', 'Gestion et Finance', 'Gestion d entreprise');

INSERT INTO etudiants (cne, nom, prenom, email,
                       date_naissance, filiere_id, moyenne) VALUES
('E2024001', 'ALAMI', 'Ali', 'ali@upf.ma', '2002-03-15', 1, 15.75),
('E2024002', 'BENANI', 'Fatima', 'fatima@upf.ma', '2001-07-22', 1,
    16.50),
('E2024003', 'CHERKAOUI', 'Youssef', 'youssef@upf.ma', '2003-01-10',
    2, 12.25),
('E2024004', 'DRISSI', 'Sara', 'sara@upf.ma', '2002-11-05', 1, 17.00)
    ,
('E2024005', 'EL FASSI', 'Karim', 'karim@upf.ma', '2001-09-18', 3,
    9.50);
```

Important

Points clés : `AUTO_INCREMENT` génère les IDs automatiquement. `FOREIGN KEY` lie étudiants aux filières. `utf8mb4` supporte tous les caractères Unicode. `ON DELETE SET NULL` : si une filière est supprimée, le `filiere_id` devient NULL.

3 Connexion à MySQL avec PDO

Définition

La connexion PDO nécessite trois paramètres : le **DSN** (*Data Source Name*), le **nom d'utilisateur** et le **mot de passe**.

Exemple – Connexion avec options recommandées

```php
<?php
try {
    $pdo = new PDO(
        "mysql:host=localhost;dbname=upf_etudiants;charset=utf8mb4",
        "root", "",
        [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES => false,
        ]
    );
    echo "Connexion réussie !";
} catch (PDOException $e) {
    die("Erreur : " . $e->getMessage());
}
?>
```

Pourquoi ? – Pourquoi ces options ?

- `ERRMODE_EXCEPTION` : lance une exception en cas d'erreur SQL (indispensable pour le débogage).
- `FETCH_ASSOC` : résultats en tableaux associatifs (`$row['nom']`).
- `EMULATE_PREPARES => false` : vraies requêtes préparées MySQL, meilleure sécurité.

Exemple – config/database.php (fichier réutilisable)

```php
<?php
define('DB_HOST', 'localhost');
define('DB_NAME', 'upf_etudiants');
define('DB_USER', 'root');
define('DB_PASS', '');

function getConnexion(): PDO {
    static $pdo = null;
    if ($pdo === null) {
        $dsn = "mysql:host=" . DB_HOST . ";dbname=" . DB_NAME
             . ";charset=utf8mb4";
        try {
```

```
            $pdo = new PDO($dsn, DB_USER, DB_PASS, [
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES => false,
            ]);
        } catch (PDOException $e) {
            die("Connexion impossible : " . $e->getMessage());
        }
    }
    return $pdo;
}
?>
```

Important

Le mot-clé `static` garantit qu'une **seule connexion** est créée même si la fonction est appelée plusieurs fois (**pattern Singleton**).

4 Les Opérations CRUD avec PDO

Définition

CRUD : **C**reate (INSERT), **R**ead (SELECT), **U**pdate (UPDATE), **D**elete (DELETE) — les quatre opérations fondamentales sur une base de données.

4.1 CREATE – Insérer des données

Sécurité – L'injection SQL

Ne jamais concaténer des variables dans une requête SQL ! Si un utilisateur entre `'; DROP TABLE etudiants; -` comme nom, la table entière est supprimée. Utilisez **toujours** des requêtes préparées.

Exemple – Insertion sécurisée avec marqueurs nommés

```php
<?php
require_once 'config/database.php';
$pdo = getConnexion();

$sql = "INSERT INTO etudiants (cne, nom, prenom, email,
                               filiere_id, moyenne)
        VALUES (:cne, :nom, :prenom, :email, :filiere, :moyenne)";

$stmt = $pdo->prepare($sql);
$stmt->execute([
    ':cne'     => 'E2024010',
    ':nom'     => 'BENNANI',
    ':prenom'  => 'Omar',
    ':email'   => 'omar.bennani@upf.ma',
    ':filiere' => 1,
    ':moyenne' => 14.50
]);

echo "ID généré : " . $pdo->lastInsertId();
?>
```

Pourquoi ? – Comment fonctionnent les requêtes préparées ?

Deux étapes séparées : **prepare()** envoie le SQL à MySQL (les marqueurs sont des emplacements vides). **execute()** envoie les valeurs **séparément**, traitées comme des données pures, jamais comme du code SQL.

4.2 READ – Lire des données

Exemple – Lister tous les étudiants avec jointure

```php
<?php
require_once 'config/database.php';
$pdo = getConnexion();

$sql = "SELECT e.*, f.nom AS filiere_nom
        FROM etudiants e
        LEFT JOIN filieres f ON e.filiere_id = f.id
        ORDER BY e.nom ASC";

$stmt = $pdo->query($sql);
$etudiants = $stmt->fetchAll();

foreach ($etudiants as $etu) {
    echo $etu['nom'] . " " . $etu['prenom']
       . " - " . $etu['filiere_nom']
       . " (" . $etu['moyenne'] . "/20)<br>";
}
?>
```

Exemple – Chercher un étudiant par ID

```php
<?php
require_once 'config/database.php';
$pdo = getConnexion();

$id = isset($_GET['id']) ? (int)$_GET['id'] : 0;

$sql = "SELECT e.*, f.nom AS filiere_nom
        FROM etudiants e
        LEFT JOIN filieres f ON e.filiere_id = f.id
        WHERE e.id = :id";

$stmt = $pdo->prepare($sql);
$stmt->execute([':id' => $id]);
$etudiant = $stmt->fetch();

if ($etudiant) {
    echo "<h2>{$etudiant['prenom']} {$etudiant['nom']}</h2>";
    echo "<p>Filière : {$etudiant['filiere_nom']}</p>";
    echo "<p>Moyenne : {$etudiant['moyenne']}/20</p>";
} else {
    echo "Étudiant non trouvé.";
}
?>
```

Exemple – Recherche avec LIKE

```php
<?php
$recherche = trim($_GET['q'] ?? '');
if (!empty($recherche)) {
    $sql = "SELECT * FROM etudiants
            WHERE nom LIKE :terme OR prenom LIKE :terme";
    $stmt = $pdo->prepare($sql);
    $stmt->execute([':terme' => "%$recherche%"]);
    $resultats = $stmt->fetchAll();
    echo count($resultats) . " résultat(s).";
}
?>
```

4.2.1 Les modes de récupération (Fetch Modes)

Mode	Description	Accès
FETCH_ASSOC	Tableau associatif	$row['nom']
FETCH_NUM	Tableau numérique	$row[0]
FETCH_OBJ	Objet anonyme	$row->nom
FETCH_BOTH	Associatif + numérique	Les deux

4.3 UPDATE – Modifier des données

Exemple – Modifier un étudiant

```php
<?php
require_once 'config/database.php';
$pdo = getConnexion();

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $sql = "UPDATE etudiants
            SET nom = :nom, prenom = :prenom,
                email = :email, moyenne = :moyenne
            WHERE id = :id";

    $stmt = $pdo->prepare($sql);
    $stmt->execute([
        ':nom'     => htmlspecialchars(trim($_POST['nom'])),
        ':prenom'  => htmlspecialchars(trim($_POST['prenom'])),
        ':email'   => filter_var($_POST['email'],
           FILTER_VALIDATE_EMAIL),
        ':moyenne' => (float)$_POST['moyenne'],
        ':id'      => (int)$_POST['id']
    ]);

    echo ($stmt->rowCount() > 0) ? "Modifié !" : "Aucun changement.";
}
?>
```

4.4 DELETE – Supprimer des données

Exemple – Supprimer un étudiant

```php
<?php
require_once 'config/database.php';
$pdo = getConnexion();

$id = isset($_GET['id']) ? (int)$_GET['id'] : 0;
if ($id > 0) {
    $check = $pdo->prepare("SELECT nom FROM etudiants WHERE id = :id");
    $check->execute([':id' => $id]);
    $etu = $check->fetch();

    if ($etu) {
        $stmt = $pdo->prepare("DELETE FROM etudiants WHERE id = :id");
        $stmt->execute([':id' => $id]);
        echo "{$etu['nom']} supprimé.";
    } else {
        echo "Étudiant introuvable.";
    }
}
?>
```

5 Requêtes Préparées : bindValue() vs bindParam()

Exemple – Différence fondamentale

```php
<?php
$stmt = $pdo->prepare("INSERT INTO etudiants (cne, nom, filiere_id)
                       VALUES (:cne, :nom, :filiere)");

// bindValue() : copie la valeur au moment de l'appel
$stmt->bindValue(':cne', 'E2024020', PDO::PARAM_STR);
$stmt->bindValue(':filiere', 1, PDO::PARAM_INT);

// bindParam() : lie par RÉFÉRENCE à une variable
$nom = "TAZI";
$stmt->bindParam(':nom', $nom, PDO::PARAM_STR);
$nom = "IDRISSI"; // Changement APRÈS le bind
$stmt->execute(); // Utilise "IDRISSI" (pas "TAZI") !
?>
```

Astuce

Utilisez `execute([])` pour les cas simples. Utilisez `bindValue()` pour typer les paramètres (`PDO::PARAM_INT` pour LIMIT/OFFSET). `bindParam()` est utile dans les boucles.

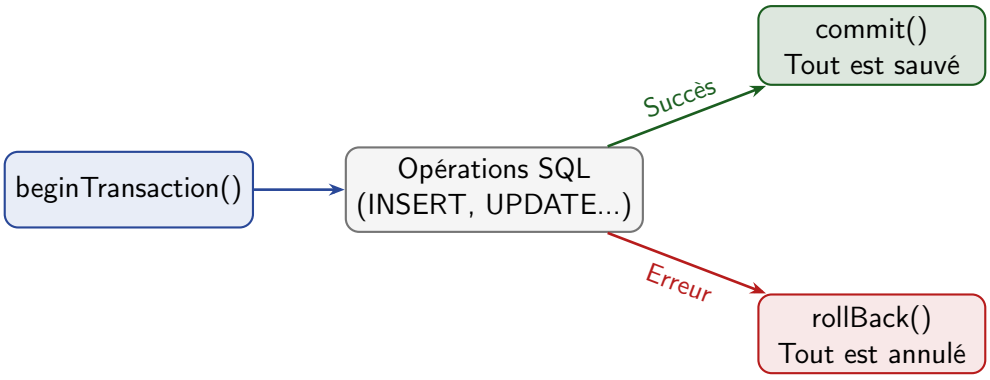
6 Les Transactions

Définition

Une **transaction** est un ensemble d'opérations SQL qui s'exécutent **entièrement ou pas du tout**. Si une opération échoue, tout est annulé (*rollback*).

Exemple – Inscription complète (compte + étudiant)

```php
<?php
require_once 'config/database.php';
$pdo = getConnexion();

try {
    $pdo->beginTransaction();

    // 1. Créer le compte utilisateur
    $sqlUser = "INSERT INTO utilisateurs (nom, email, mot_de_passe)
                VALUES (:nom, :email, :mdp)";
    $stmtUser = $pdo->prepare($sqlUser);
    $stmtUser->execute([
        ':nom'   => 'ALAOUI Amine',
        ':email' => 'amine@upf.ma',
        ':mdp'   => password_hash('Mdp2024!', PASSWORD_DEFAULT)
    ]);

    // 2. Créer l'étudiant
    $sqlEtu = "INSERT INTO etudiants (cne, nom, prenom, email,
                                      filiere_id)
               VALUES (:cne, :nom, :prenom, :email, :filiere)";
    $stmtEtu = $pdo->prepare($sqlEtu);
    $stmtEtu->execute([
        ':cne'     => 'E2024030',
        ':nom'     => 'ALAOUI',
        ':prenom'  => 'Amine',
        ':email'   => 'amine.alaoui@upf.ma',
        ':filiere' => 1
    ]);

    // Tout OK : valider
    $pdo->commit();
    echo "Inscription complète réussie !";

} catch (PDOException $e) {
    // Erreur : annuler TOUT
    $pdo->rollBack();
    echo "Erreur : " . $e->getMessage();
}
?>
```


commit()
Tout est sauvé
Succès
beginTransaction()
Opérations SQL
(INSERT, UPDATE...)
Erreur
rollBack()
Tout est annulé

7 Pagination des Résultats

Définition

La **pagination** divise les résultats en **pages** pour éviter d'afficher des centaines de lignes. On utilise `LIMIT` et `OFFSET` en SQL, combinés avec `$_GET` pour la navigation.

Exemple – Pagination complète

```php
<?php
require_once 'config/database.php';
$pdo = getConnexion();

$parPage = 5;
$page = isset($_GET['page']) ? max(1, (int)$_GET['page']) : 1;
$offset = ($page - 1) * $parPage;

// Compter le total
$total = $pdo->query("SELECT COUNT(*) FROM etudiants")
             ->fetchColumn();
$totalPages = ceil($total / $parPage);

// Récupérer la page courante
$sql = "SELECT e.*, f.nom AS filiere_nom
        FROM etudiants e
        LEFT JOIN filieres f ON e.filiere_id = f.id
        ORDER BY e.nom
        LIMIT :limit OFFSET :offset";

$stmt = $pdo->prepare($sql);
$stmt->bindValue(':limit', $parPage, PDO::PARAM_INT);
$stmt->bindValue(':offset', $offset, PDO::PARAM_INT);
$stmt->execute();
$etudiants = $stmt->fetchAll();
?>

<h2>Page <?= $page ?>/<?= $totalPages ?></h2>
<!-- Affichage du tableau des étudiants -->

<div class="pagination">
    <?php if ($page > 1): ?>
        <a href="?page=<?= $page - 1 ?>">Précédent</a>
    <?php endif; ?>

    <?php for ($i = 1; $i <= $totalPages; $i++): ?>
        <?php if ($i == $page): ?>
            <strong>[<?= $i ?>]</strong>
        <?php else: ?>
            <a href="?page=<?= $i ?>"><?= $i ?></a>
```

```
        <?php endif; ?>
    <?php endfor; ?>

    <?php if ($page < $totalPages): ?>
        <a href="?page=<?= $page + 1 ?>">Suivant</a>
    <?php endif; ?>
</div>
```

8 Système d'Authentification avec PDO

Combinons les sessions et PDO pour un système de connexion sécurisé.

8.1 Inscription sécurisée

Exemple – inscription.php

```php
<?php
session_start();
require_once 'config/database.php';
$erreurs = [];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $pdo = getConnexion();
    $nom = htmlspecialchars(trim($_POST['nom'] ?? ''));
    $email = filter_var($_POST['email'] ?? '',
                        FILTER_VALIDATE_EMAIL);
    $mdp = $_POST['mdp'] ?? '';
    $mdpConfirm = $_POST['mdp_confirm'] ?? '';

    if (empty($nom)) $erreurs[] = "Nom obligatoire.";
    if (!$email) $erreurs[] = "Email invalide.";
    if (strlen($mdp) < 8) $erreurs[] = "Mot de passe trop court.";
    if ($mdp !== $mdpConfirm) $erreurs[] = "Mots de passe différents.
        ";

    // Vérifier unicité de l'email
    if ($email) {
        $check = $pdo->prepare(
            "SELECT id FROM utilisateurs WHERE email = :email");
        $check->execute([':email' => $email]);
        if ($check->fetch()) {
            $erreurs[] = "Email déjà utilisé.";
        }
    }

    if (empty($erreurs)) {
        $sql = "INSERT INTO utilisateurs (nom, email, mot_de_passe)
                VALUES (:nom, :email, :mdp)";
        $stmt = $pdo->prepare($sql);
        $stmt->execute([
            ':nom'   => $nom,
            ':email' => $email,
            ':mdp'   => password_hash($mdp, PASSWORD_DEFAULT)
        ]);
        header("Location: login.php?succes=1");
        exit;
    }
```

```
}
?>
```

8.2 Connexion sécurisée

Exemple – login.php avec PDO

```
<?php
session_start();
require_once 'config/database.php';
$erreur = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $pdo = getConnexion();
    $email = filter_var($_POST['email'] ?? '',
                        FILTER_VALIDATE_EMAIL);
    $mdp = $_POST['mdp'] ?? '';

    if ($email) {
        $stmt = $pdo->prepare(
            "SELECT * FROM utilisateurs WHERE email = :email");
        $stmt->execute([':email' => $email]);
        $user = $stmt->fetch();

        // Vérifier le hash du mot de passe
        if ($user && password_verify($mdp,
                                     $user['mot_de_passe'])) {
            session_regenerate_id(true);
            $_SESSION['connecte']  = true;
            $_SESSION['user_id']   = $user['id'];
            $_SESSION['user_nom']  = $user['nom'];
            $_SESSION['user_role'] = $user['role'];

            header("Location: dashboard.php");
            exit;
        } else {
            $erreur = "Email ou mot de passe incorrect.";
        }
    }
}
?>
```

Important

Sécurité de l'authentification :

- `password_hash()` : hache avec bcrypt (irréversible).
- `password_verify()` : compare le saisi avec le hash.

- **Ne jamais** stocker les mots de passe en clair !
- **Ne jamais** dire si c'est l'email ou le mot de passe qui est faux.
- `session_regenerate_id(true)` : protège contre le vol de session.

9 Récapitulatif des Méthodes PDO

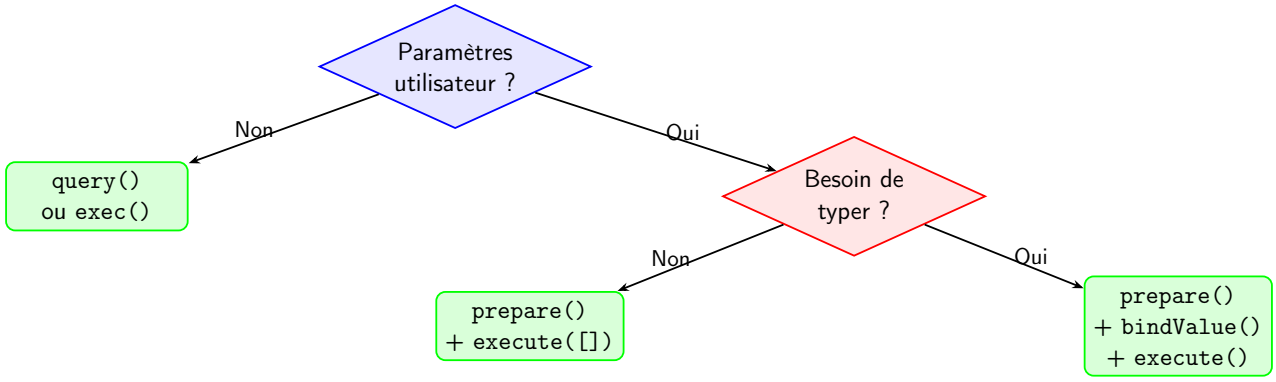
9.1 Méthodes de l'objet PDO

Méthode	Description
`query($sql)`	Exécute une requête sans paramètres (SELECT)
`prepare($sql)`	Prépare une requête pour exécution ultérieure
`exec($sql)`	Exécute sans résultats (INSERT, UPDATE, DELETE simples)
`lastInsertId()`	Retourne l'ID du dernier INSERT
`beginTransaction()`	Démarre une transaction
`commit()`	Valide la transaction
`rollBack()`	Annule la transaction

9.2 Méthodes de PDOStatement

Méthode	Description
`execute([$params])`	Exécute la requête préparée
`fetch($mode)`	Récupère la prochaine ligne
`fetchAll($mode)`	Récupère toutes les lignes
`fetchColumn($n)`	Récupère une seule colonne
`rowCount()`	Nombre de lignes affectées (UPDATE/DELETE)
`bindValue($p, $v, $t)`	Lie une valeur à un paramètre
`bindParam($p, $var, $t)`	Lie une variable par référence

9.3 Schéma décisionnel



10 Bonnes Pratiques et Erreurs Courantes

Sécurité – Les 8 règles d'or

1. **Toujours** utiliser des **requêtes préparées** pour les données utilisateur.
2. **Toujours** valider et nettoyer les entrées (`htmlspecialchars`, `filter_var`).
3. **Toujours** hacher les mots de passe (`password_hash`).
4. **Toujours** utiliser `try/catch` pour les opérations PDO.
5. **Toujours** centraliser la connexion dans un fichier unique.
6. **Jamais** afficher les erreurs SQL en production.
7. **Jamais** concaténer des variables dans une requête SQL.
8. **Jamais** stocker les mots de passe en clair.

10.1 Erreurs courantes

Mauvaise pratique	Bonne pratique
`"... WHERE id = $id"`	`prepare("... WHERE id = :id")`
`die($e->getMessage())` en prod	Logger l'erreur + message générique
`SELECT *` systématiquement	Sélectionner les colonnes nécessaires
Nouvelle connexion par requête	Connexion unique (singleton)
Mot de passe en clair	`password_hash($mdp, PASSWORD_DEFAULT)`
Faire confiance à `$_GET['id']`	`(int)$_GET['id']` ou `filter_var()`

10.2 Structure recommandée du projet

Exemple – Arborescence

```
gestion-etudiants/
|-- config/
|   |-- database.php          (connexion PDO)
|-- includes/
|   |-- header.php             (en-tete + navigation)
|   |-- footer.php             (pied de page)
|-- css/
|   |-- style.css              (feuille de style UPF)
|-- uploads/photos/             (photos etudiants)
|-- index.php                  (accueil)
|-- login.php                  (connexion)
|-- inscription.php            (inscription)
|-- logout.php                 (deconnexion)
|-- etudiants/
|   |-- liste.php              (liste + pagination)
```

```
|   |-- ajouter.php          (formulaire d'ajout)
|   |-- modifier.php         (modification)
|   |-- supprimer.php        (suppression)
|   |-- detail.php           (fiche etudiant)
|-- filieres/
|   |-- liste.php            (liste des filieres)
|-- dashboard.php            (tableau de bord)
```

Astuce – Pour aller plus loin

Ce cours couvre les fondamentaux de PDO. Pour approfondir :

- **Architecture MVC** : séparer Modèle, Vue et Contrôleur.
- **ORM** : Doctrine, Eloquent (abstraction objet de la BDD).
- **Frameworks** : Laravel, Symfony.
- **API REST** : services web avec PDO et JSON.